

운영자 지침서

프로젝트 코드	
작 성 자	
검 토 자	
문 서 번 호	
버 전	V1.2
작 성 일 자	2026. 03. 09

Copyright

당 팀의 사전 승인없이 본 내용의 전부 또는 일부에 대한
복사, 전재, 배포 사용을 금합니다.

원웨이시스템

제.개정내역

버전/릴리스	작성일자	개요
1.0	2022.01.03	제정
1.01	2024.05	미래엣셋
1.02	2023.10	메리츠증권
1.03	2024.05	NH
1.1	2025.11	KB Bank
1.2	2026.03	IBK Bank

대체 인도물

본 문서는 다음의 인도물을 대체한다:

[illegible]

목 차

1. 개요	
1.1. 범위 및 목적	
1.2. 전제	
1.3. 문서의 구성	
2. 시스템 개요.....	
2.1. 시스템 구성요소	
2.1.1. 하드웨어 사양.....	
2.2. 프로세스 구성	
2.2.1. 개요.....	
2.2.2. 기본 환경.....	
2.2.3. 프로세스 기본규칙.....	
2.2.4. Directory 구성	
2.2.5. Shared Memory (SHM) Segment 구조.....	
2.2.6. 버전 관리.....	
2.2.7. 보안 관리.....	
3. 유지보수 작업 개요	
3.1. 프로그램 수정 및 신규 작성.....	
3.1.1. 신규 program 작성.....	
3.1.2. 기존 program으로 신규 process 생성.....	
3.1.3. make 시 compile 기준.....	
3.2. PROCESS의 기동과 종료.....	
3.2.1. 전 process의 기동.....	
3.2.2. 전 process의 종료.....	
3.2.3. 한 부문의 업무 process들만 종료.....	
3.2.4. 종료된 업무 process들을 다시 기동.....	
3.2.5. 업무 process 하나만 종료.....	
3.2.6. 업무 process 하나만 종료했다가 바로 재 기동.....	
3.2.7. 부문 daemon의 강제 종료 및 실행	
3.3. DATA 및 LOG의 정리 (삭제)	
3.4. LOG 처리 및 MONITORING	
3.5. 주요 운영 PROCESS	
4. PROCESS 운영 시간	
4.1. REAL SYSTEM 운영 시간	
5. 운영 UTILITY SHELL 사용법.....	
5.1. 조회 관련 SHELL 사용법	
5.2. 개발 관련 SHELL 사용법	
5.3. 변경 관련 SHELL 사용법	

1. 개 요

1.1. 범위 및 목적

- 해당 시스템은 IBK Bank 의 “OMS 시스템 구현”을 목적으로 개발 되었다.
- 본 문서는 “자동 주문 시스템 구현” 개발 프로젝트를 통해 구축된 제품의 유지보수 및 적용시스템을 담당할 요원들을 위한 문서이다.
- 본 문서는 유지보수 요원들이 “자동 주문 시스템 구현”의 운영 및 유지보수를 수행하기 위해 필요한 정보를 제공하는 것을 목적으로 한다.

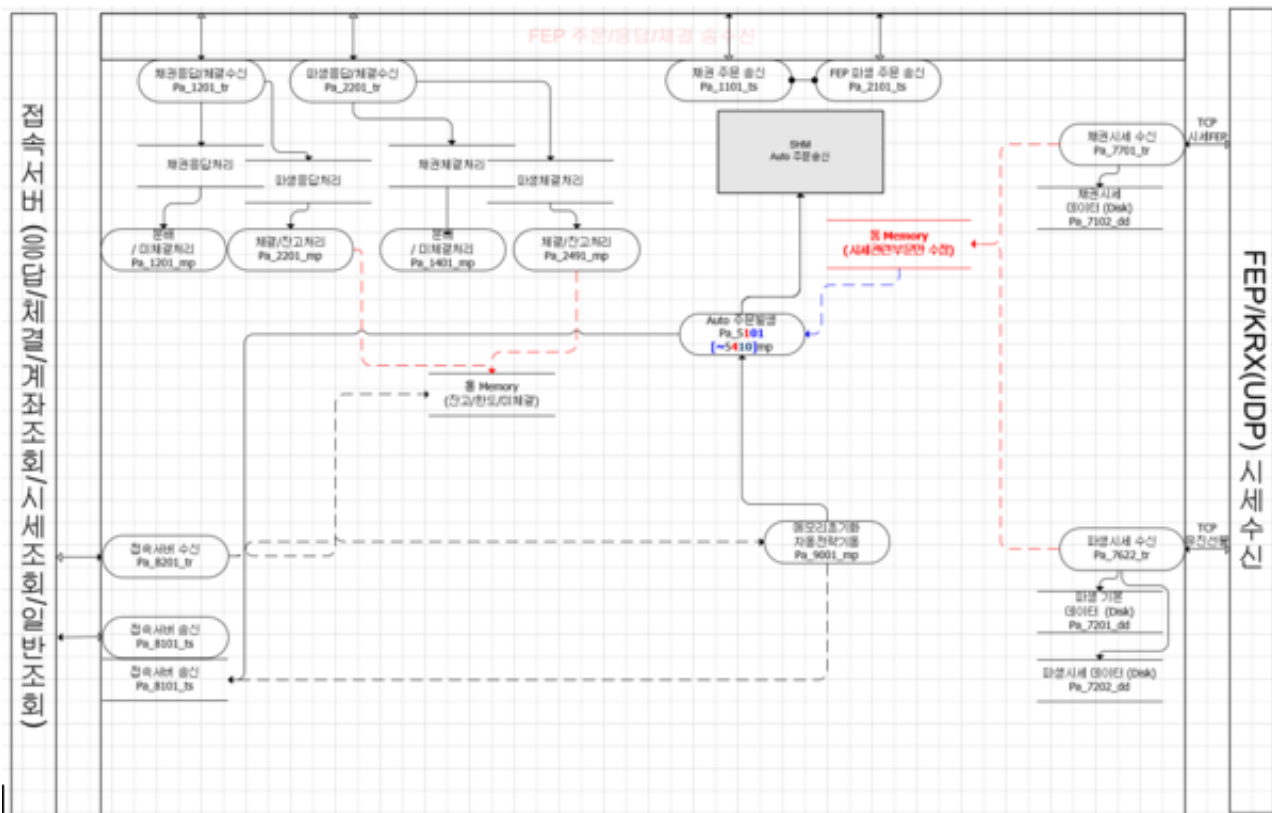
1.2. 전제

- 본 문서는 유지보수 요원들을 위한 문서로 작성되었으며, “자동 주문 시스템 구현” 개발 및 유지보수 요원과 시스템 기반환경을 운영할 유지보수 요원 모두를 대상으로 한다.
- 본 문서는 유지보수 및 운영에 필요한 개괄적인 정보 및 지침을 제공하는 것을 목적으로 하며 구체적인 정보는 요소별로 별도의 경로를 통해 전달하는 것을 원칙으로 한다.

1.3. 문서의 구성

본 문서는 아래와 같이 구성되어 있다.

- 2장 : 시스템 개요
- 3장 : 유지보수 작업 개요
- 4장 : Process 운영 시간
- 5장 : 회선 구성
- 6장 : 운영 Utility Shell 사용법
- 7장 : 운영메뉴



2.2.2. 기본 환경

P	Test Server	Real Server
Home Directory	\$HOME/st01	\$HOME/st01
Data Directory	\$HOME/st02/DAT	\$HOME/st02/DAT
FIFO Directory	\$HOME/st02/FIFO	\$HOME/st02/FIFO
Log Directory	\$HOME/st03/LOG	\$HOME/st03/LOG
환경변수 File	\$HOME/st01/env/pkg_env.sh	\$HOME/st01/env/pkg_env.sh

2.2.3. 프로세스 기본규칙

A. 프로세스 ID 구성 (10 ~ 13 bytes)

p	a	_	1	1	0	1	_	t	s
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)

<항목 설명>

번호	항목	표현방식	설명	사용형태
(1) ~ (2)	부문코드	A2	* 부문코드 참조	고정
(3)	연결자	A1	"_"	고정
(4) ~ (7)	업무구분코드	N4	* 업무구분코드 참조	고정
(8)	연결자	A1	"_"	고정
(9) ~ (10)	업무처리코드	A2	* 업무처리코드 참조	고정

1. 부문코드 (2) + 업무구분코드 (4) + 업무처리코드 (2)

➔ header 의 업무구분식별자 (대문자로 표시)

2. 업무 프로세스명은 기본 10 자리이고 최대 13 자리까지 사용

<용례 1> pa_8101_ts (접속서버송신 process)

- 1) 부문: "pa"
- 2) 연결자: "_"
- 3) 업무구분코드: "8101"
- 4) 연결자: "_"
- 5) 업무처리코드: "ts"
- 6) 업무구분식별자: "PA8101TS"

<용례 2> pa_8201_tr (접속서버수신 process)

- 1) 부문 : "pa"

- 2) 연결자: "_"
- 3) 업무구분코드: "8201"
- 4) 연결자: "_"
- 5) 업무처리코드: "tr"
- 6) 업무구분식별자: "PA8201TR"

B. 부문코드 구성 (2 bytes)

번호	항목	표현방식	항목설명
1	시스템구분	A1	P:시스템구분
2	업무구분	A1	a: 시스템트레이딩 b ~ v: 예비 x: utility z: 관리

C. 업무구분코드 구성 (4 bytes)

번호	항목	표현방식	항목설명	사용코드
1	대분류 (상품구분)	N1	- 업무구분 - 1 : 주문/응답/체결 관련 5 : 자동전략 6 : 배치/조회, 시세수신 7 : 시세수신 8 : 접속서버 송/수신 9 : 메모리 초기화, 자동전략 기동	0 ~ 9
2	중분류 (기능구분)	N1	- 주문체결 → 1:주문, 2:주문확인, 4:체결	0 ~ 9
3	소분류 또는 일련 번호	N1	- 업무에 따라 1) 또는 2) 선택 1) 소분류: 한 상품 내 상세 분류 2) 일련번호: 번호 3-4 두 자리를 일련번호로 이용	0 ~ 9
4	일련번호	N1	- 각 업무 세부 구분별 일련번호 - e.g. KRX 대외송신 → 1:1 송신, 2:2 송신, 3:3 송신...	0 ~ 9

D. 업무처리코드 구성 (2 bytes)

번호	항목	표현방식	항목설명
1	업무처리구분 1	A1	- m:관리 (Manager, Auto Order) - d:분배 (Divide) - p:X.25, t:TCP/IP, q:Tuxedo, b:DB
2	업무처리구분 2	A1	- p:처리 (Process) - d:분배 (Division) - r:수신 (Receive), s:송신 (Send), a:송수신 (Send and Receive)

<업무처리코드 상세>

mp: Manager Process / Auto Order, dd: Divide Division, tr: TCP/IP Receive, ts: TCP/IP Send,
tr:TCP/IP Receive, cm: curses 운영메뉴

2.2.4. Directory 구성

기능	Directory	Sub Dir	업무	Alias
/mtzsw/mzdd/st01/				
실행 File	bin		실행 module	pbin
SHM 환경	cfg		shared memory 환경 설정	pcfg
공통 환경	env		environment, alias	pnve
Header	inc		header file	pinc
Library	lib		library	plib
Makefile	make		makefile	pmake
		PA ~ PZ	source makefile	pamake ~ pzmake
		SUB	공통 makefile	pmakes
Object	obj		object file	pobj
		PA ~ PZ	부문별 object	paobj ~ pzobj
		SUB	공통 object	pobjs
Shell	shl		shell script	pshl
Source Program	src		source program file	psrc
		PA	Business Source	pasrc
		PX	Utility	pxsrc
		PZ	관리	pzsrc
공통함수	sub		공통	psub
공통관리	utl		source, 공통 utility, 개인작업	putl
/mtzsw/mzdd/st02/				
Data	DAT		data file	pdat
		PA ~ PZ	부문별 data	padat ~ pzdat
FIFO	FIFO		FIFO file	pfifo
			부문별 FIFO	pafifo ~ pzfifo
/mtzsw/mzdd/st03/				
Log	LOG		log file	plog
		PA ~ PZ	부문별 log	palog ~ pzlog

2) 전체 daemon SHM (INFO)

2.1) SHM key: 0x21000000L (SUB_SHM_KEY)

2.2) segment 구조

(a)	(b)	...	(z)
(1)	(2)	(3)	
D_K = 0	D_K = 1	D_K = 2	

-. (a) → PA 부문 영역, (b) → PB 부문 영역, ..., (z) → PZ 부문 영역

-. (1), (2), ... → SHM_All_Daemon_Info[D_K] = INFO(D_K)

-. (1) → SHM_All_Daemon_Info = SHM_All_Daemon_Info[0] = INFO(0)

-. (2) → SHM_All_Daemon_Info[1] = INFO(1)

-. (a)의 크기 = (b)의 크기 = (z)의 크기 = ALL_DAEMON_INFO 의 크기 = 136 bytes

3) 부문 SHM

3.1) SHM key

-. 0x21010000L (PA), 0x21020000L (PB), ..., 0x21230000L (PW)

3.2) segment 구조

(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
(A)											

-. Shm_Mem[D_K].start_FIFO_fd = SFIFD(D_K) = START_FD

-. Shm_Mem[D_K].exit_FIFO_fd = EFIFD(D_K) = EXIT_FD

-. Shm_Mem[D_K].daemon_FIFO_fd = DFIFD(D_K) = DTART_FD

-. (1) → SHM_Mem[D_K] → Shm_Mem[D_K].Daemon[0] = DAEMON(D_K)

-. (2) → Shm_Mem[D_K].Proc[0] = PROC(D_K,0)

-. (A) → Shm_Mem[D_K].Proc[P_K] = PROC(D_K,P_K)

-. (3) → Shm_Mem[D_K].File[0] = FILEM(D_K,0)

-. (4) → Shm_Mem[D_K].DShm[0] = DSHM(D_K,0)

-. (5) → Shm_Mem[D_K].Queue[0] = QUEUE(D_K,0)

-. (6) → Shm_Mem[D_K].Tuxedo[0] = TUXEDO(D_K,0)

-. (7) → Shm_Mem[D_K].SendTr[0] = SENDTR(D_K,0)

-. (8) → Shm_Mem[D_K].RecvTr[0] = RECVTR(D_K,0)

-. (9) → Shm_Mem[D_K].Lcn[0] = LCN(D_K,0)

-. (10) → Shm_Mem[D_K].Tcp1[0] = TCP1(D_K,0)

-. (11) → Shm_Mem[D_K].Tcp2[0] = TCP2(D_K,0)

-. (12) → Data_Ptr[0] → PROC(D_K,P_K).data = SHM_DATA

- (a)의 크기 = SUB_DAEMON_INFO
- (b)의 크기 = PROCESS_INFO * INFO[D_K].process_count
- (c)의 크기 = FILE_INFO * INFO[D_K].file_count
- (d)의 크기 = DSHM_INFO * INFO[D_K].dshm_count
- (e)의 크기 = QUEUE_INFO * INFO[D_K].queue_count
- (f)의 크기 = TUXEDO_INFO * INFO[D_K].tuxedo_count
- (g)의 크기 = LCN_INFO * INFO[D_K].lcn_count
- (h)의 크기 = TCP1_INFO * INFO[D_K].tcp1_count
- (i)의 크기 = TCP2_INFO * INFO[D_K].tcp2_count
- (j)의 크기 = SHM_DATA_SIZE * INFO[D_K].data_count
- cf. SHM_DATA_SIZE = 5,120

4) data SHM segment 및 semaphore 의 key 구조

0x(1)(2)(3)(4)(5)(6)(7)(8) ->

(5)(6) - 00:부문 SHM, 01~50:data SHM segment 의 base key

51~99:index SHM segment 의 base key

(7)(8) - 00:data SHM, 01~99:semaphore

5) 부문별 관리 FIFO 에 대한 설명

- START_FD (FIFO 명 = PA[~z]_FIFO)

- 기본 poll 처리를 위한 용도. Fifo_Event_Rtn () 처리.

file, socket 등 이외에 기본 timeout 처리를 위한 용도로 쓰임

- EXIT_FD (FIFO 명 = PA[~z]_FIFO_exit)

- user 가 pz_fepp_mp 를 SIGTERM 으로 kill (pz_fepp_mp 만 restart)

- pz_fepp_mp 는 기동 시 어떤 부문 daemon 이 기동 중이면 해당 부문의 exit FIFO 에 '1'을 write 하고 그 부문 daemon 에 USR1 signal 을 줌

- 부문 daemon 은 USR1 signal 을 받고 EXIT_FD 에 input 이 있으면 INFO SHM 을 detach 한 후 다시 attach 함

- user 가 pz_fepp_mp 를 USR2 로 kill (전체 process 종료)

- pz_fepp_mp 는 USR2 signal 을 받으면 현재 기동 중인 모든 부문의 exit FIFO 에 '1'을 write 하고 해당 부문 daemon 에게 USR2 signal 을 줌

- 부문 daemon 은 USR2 signal 받으면 해당 부문의 업무 process 들을 종료시키고 EXIT_FD 에 input 있으면 자기 자신도 종료함

- DTART_FD (FIFO 명 = pa[~z]_FIFO_ctrl)

- pz_fepp_mp: DTART_FD (pz_FIFO_ctrl)에 poll 걸고 3 초 간격으로 User_Signal (), Start_Process ()

처리를 함. pz_FIFO_ctrl 에 write 하는 경우는 없으며 단지 poll 처리를 위한 용도로만 쓰임

- 부문 daemon: pz_fepp_mp 와 마찬가지로 해당 부문의 DTART_FD 에 poll (3 초 간격) 걸고

User_Signal (), Start_Process () 처리함.

업무 process 들은 종료 시 해당 부문의 DTART_FD 에 '1'을 write 함으로써 부문 daemon 이

걸고 있는 DTART_FD 에 pollin 발생 하여 바로 signal, start 처리하러 가도록 함

2.2.5. 버전 관리

1. system OS 및 software 는 주기적으로 tape 에 backup 하여 보관
2. source program 은 주기적으로 tape 에 backup 하여 보관
 - backup 대상: /mtzsw/mzdd/st01
 - program 변경 혹은 신규 program 작성 시 형상관리 tool 의 적용을 받아 source 의 version 관리를 하도록 함
3. 관리하는 data 와 log 는 기본적으로 8 일분 이상을 기본 data 와 log directory 에 보관할 것을 권장하며 보관일수는 조정 가능함
 - data: /mtzsw/mzdd/st02/DAT/<부문명>/<일자>/
 - log: /mtzsw/mzdd/st03/LOG/<부문명>/<일자>/

2.2.6. 보안 관리

보안관리는 별도의 보안관리 tool 을 사용하지 않고 아래와 같이 system 과 process 측면에서 관리함

구분	항목	내용
system	login 시 check	- login 시 login ID, password, 사용자 name 을 check 함 - password 는 FEP 담당자의 판단으로 수시로 변경하여 사용
Process	업무계 접속	- 업무계 server 는 지정된 대외계 server 의 IP address 와 port 로 접속하여 대외계시스템과 전문을 송수신함 - IP address 및 port 변경 시 상호간에 반드시 통보요함

3. 유지 보수 작업 개요

3.1. 프로그램 수정 및 신규 작성

3.1.1. 신규 program 작성

- 별도의 업무를 수행하는 신규 process 를 추가할 경우임

1) 업무에 맞는 신규 program 을 작성

- DATA_SIZE 는 file.ini (data SHM 을 사용 시 dshm.ini)의 data size 와 맞춤
- 관련 program (분배 등) 추가 작성 및 수정 검토
- 기타 공통함수 (~/sub), 관리 (~/src/PZ), utility (~/src/PX), shell (~/shl)등의 추가/수정 여부 검토

2) 신규 program 이 생성하는 process 에 따라 환경설정 file (~cfg/*.ini)을 수정

- 접속 line 관련 추가 사항 발생 시 관련 file 수정 (tcp1.ini, tcp2.ini)
- 신규로 생성한 process 가 read 혹은 write 할 file 이 추가로 발생한 경우 file.ini 에 추가
 - file 명은 해당 file 을 read 하는 process 와 동일하게 정함
 - 해당 file 을 read 하는 process 가 둘 이상이고 각 process 가 해당 file 을 모두 읽으며 다른 read count 로 관리한다면 해당 process 의 수만큼 FIFO count 를 등록함
 - 둘 이상의 process 가 해당 file 을 선택적으로 (balancing) read 하여 하나의 read count 로 관리될 경우는 FIFO count 를 1 로 등록
- 신규로 생성한 process 가 read 혹은 write 할 data SHM 이 추가로 발생한 경우 dshm.ini 에 추가
 - data 명은 해당 data SHM 을 read 하는 process 와 동일하게 정함
 - 해당 data SHM 을 read 하는 process 가 둘 이상이고 각 process 가 해당 data SHM 을 모두 읽으며 다른 read count 로 관리한다면 해당 process 의 수만큼 FIFO count 를 등록함
 - 둘 이상의 process 가 해당 data SHM 을 선택적으로 (balancing) read 하여 하나의 read count 로 관리될 경우는 FIFO count 를 1 로 등록
- process 정보를 proc.ini 에 추가 (각 항목의 setting 방법은 proc.ini 의 맨 위에 있는 주석을 참고)
 - process 기본 정보 (ID, Status, Type, Start_Time 등)를 등록
 - input/output file 이 있는 경우 file.ini 와 file 관련 정보를 맞추고 통신 process 인 경우는 접속 line 관련 환경설정 file (lcn.ini 등)과 해당 정보가 일치되도록 주의 요함
 - input/output data SHM 이 있는 경우 dshm.ini 와 data SHM 관련 정보를 맞춤
- daemon.ini 의 해당 process 가 속한 부문 관련 정보를 확인하여 변경을 요하는 항목이 있으면

수정함

★ daemon.ini 의 Count 항목 (Proc_Count 등)은 해당 항목 관련 ini file 에 등록된 실제

건수보다 크게 정하되 5 의 배수가 되도록 함. 예를 들어 PA 부문의 process 수가 166 개면

proc.ini 의 PA 부문 Proc_Count 값은 166 으로 하고 daemon.ini 의 Daemon_C_Proc_Count

값을 170 으로 함

(daemon.ini 의 Count 값에 의하여 SHM 생성 시 segment 의 크기가 산정됨)

- . 환경설정 file 을 수정할 때 각 file 내의 부문별 시작 위치에 있는 부문별 Count 정보를

정확히 setting 하도록 주의함

- . 수정할 대상 환경설정 file 의 item 수에 변동이 없는 경우 (즉 proc.ini 를 예로 들면 기존

process 의 항목값만 변경하고 신규로 추가되거나 삭제되는 process 가 없는 경우) 그대로

수정해도 무방하나 만일 변동이 있는 경우는 cfgback.sh 를 이용하여 수정용 file (~.ini.tmp)

을 만든 후 이 file 을 수정한 다음 cfgload.sh 를 사용해서 수정된 내용을 원본 file (~.ini)

에 반영하는 방식으로 진행

3) 신규 program 이 생성하는 process 에 따라 필요한 경우 header file (~/inc/)을 수정

- . 주문/체결/조회 등 대외 송수신 전문과 관련된 structure 에 변동이 있는 경우 해당 부문의

structure 정의 header (pa_struct.h 등)의 해당 structure 수정

4) 신규 program 이 생성하는 process 와 관련된 makefile 을 수정

- . 해당 부문의 makefile directory (PA ~ PZ, SUB)에서 수정할 makefile 을 찾아 수정

3.1.2. 기존 program으로 신규 process 생성

1) 동일한 업무를 수행하는 신규 process 를 추가할 경우 필요 시 동종 업무를 수행하는 program 을

수정

2) DATA_SIZE 를 define 하는 부분이 추가되거나 추가로 routine 이 필요한 경우만 수정하고 그 외에

동일한 program 으로 process 만 추가되는 경우에는 '신규 program 작성' 처리 방법에 준하여 진행

3.1.3. make 시 compile 기준

1) header file (~/inc/*.h) 변경

→ 해당 header file 이 지정된 makefile (~.mk)에 의해

생성되는 binary 전체가 compile 됨

2) sub program (~/sub/*.c) 변경

→ 전체가 compile 됨

3) source program (~/src/*/*.c) 변경

→ 수정된 program 만 compile 됨

4) object file (~/obj/*/*.o) 삭제

→ 삭제된 object 의 원본 program 을 touch 하거나 binary 를

삭제해야 compile 됨

5) 실행 module (~/bin/*) 삭제

→ 삭제된 binary 만 compile 됨

★ HP 의 경우 process 가 실행 중인 상태에서 해당 process 를 만드는 원본 program 을 수정하고 compile 을 하게 되면 오류 message 가 나오며 binary 가 update 되지 않으며 Sun 의 경우는 compile 은 되지만 실행 중이던 process 에 문제를 일으킬 수 있음.

이 경우 해당 binary (실행 module)를 ~/bin/bak/ directory 로 move 한 후 compile 하면 정상적으로 compile 되고 binary 가 신규로 생성됨. 새로 compile 한 내용이 반영되게 하려면 해당 process 를 kill 시키면 됨 (psk.sh <process ID>)

리눅스의 경우 실행중인 process 도 컴파일이 가능하다. 단, 적용을 하려면 재기동 또는 익일적용된다.

3.2. PROCESS의 기동과 종료

3.2.1. 전 process의 기동

-. pbin → pz_fepp_mp

- pz_fepp_mp, pz_procchk_mp, 부문 daemon (pa[~pw]_daemon_mp)과 각 업무 process 들이 기동됨

3.2.2. 전 process의 종료

-. psk.sh pz_fepg_mp USR2: pz_fepg_mp 를 종료시킴

- pz_fepg_mp 는 부문 daemon 에 USR2 signal 을 전달하여 부문 daemon 은 해당 부문의 업무 process 를 종료시키고 자신도 종료됨. 또한 pz_procchk_mp 도 종료시킴

3.2.3. 한 부문의 업무 process들만 종료

-. psk.sh <부문 daemon 명> USR2

- e.g. psk.sh PA_daemon_mp USR2

- PA 부문의 업무 process 들만 종료되고 pa_daemon_mp 는 계속 기동 상태로 존재함

3.2.4. 종료된 업무 process들을 다시 기동

-. psk.sh <부문 daemon 명> USR1

- e.g. psk.sh pa_daemon_mp USR1

- PA_daemon_mp 은 해당 부문의 업무 process 중에 현재 기동 중인 것이 있으면 종료시키고
 부문 SHM 을 다시 구성한 후 업무 process 들을 기동시킴

-. USR1 과 USR2 는 user defined signal 로 process 의 기동과 종료를 위한 signal 처리에 이용

3.2.5. 업무 process 하나만 종료

1) setpstat.sh <process 명> 9 → psk.sh <process 명>

- e.g. setpstat.sh PA_1101_ts 9 → psk.sh PA_1101_ts

2) jstop.sh <process 명>

- e.g. jstop.sh PA_1101_ts

- 위 1)과 같은 기능

3.2.6. 업무 process 하나만 종료했다가 바로 재 기동

-. psk.sh <process 명>

- e.g. psk.sh pa_1101_ts

- 해당 process 종료 후 부문 daemon 에 의하여 재 기동됨

3.2.7. 부문 daemon의 강제 종료 및 실행

1) 부문 daemon 의 강제 종료

-. 부문 daemon 을 종료하기 전 해당 부문의 업무 process 들을 모두 종료해야 함
(e.g. psk.sh pa_dae USR2)

-. 부문 daemon 종료 방법은 다음

a) runstop.sh 을 실행

2) 종료된 부문 daemon 의 실행

-. runstop.sh 를 실행하여 기동

-. 환경설정 file 을 다시 읽어서 SHM 에 load 하거나 기존 SHM 상태 그대로 부문 daemon 만
기동할 수도 있음

-. 만일 환경설정 file 을 수정 중인 경우에는 현재 수정 중인 상태가 아직 SHM 에 반영되어서는
안되므로 반드시 기존 SHM 상태 그대로 기동해야 함

3.3. DATA 및 Log의 정리 (삭제)

- 사용법: compact.sh <보관일수> <부문명>

-. 해당 '보관일수' 이전의 자료 (data, log 등)를 삭제함

-. 용례

1) compact.sh 5 c

- PA 부문의 현 운영일자 (INFO.date)로부터 5 일 전까지의 자료만 보관하고 그 이전 일자의 모든 자료 삭제

2) compact.sh 3

- PW, PX, PY, PZ 은 현 시스템일자, 그 외의 부문은 현 운영일자 (INFO.date)를 기준으로 하여 전 부문의 3 일 전 일자 이전 자료를 정리

3) compact.sh 0

- 전 부문의 모든 data 및 log 삭제 (주의 요함). 단 00000000/ directory 는 삭제 대상에서 제외

4) compact.sh

- 전 부문의 기본 정리일수 (7 일) 이전의 모든 data 와 log 를 삭제

-. 현재 기본 보관일수 간격으로 자동 정리되도록 daemon 에서 처리

(오늘을 포함한 기본 보관일수의 data 와 log 를 기본 data, log directory 에 항상 보관)하고 있으므로 수작업으로 PZ_compact_mp 를 기동할 필요는 거의 없으나 data 나 log file system 의 사용량이 90%를 넘으면 수작업으로 정리 요함)

- ~/cfg/daemon.ini 에 지정된 기본 보관일수 (Compact_Days)를 수정하여 조정 가능 (8 일 이상 보관할 것을 권장함)

3.4. LOG 처리 및 MONITORING

1) 관련 program: ~/sub/log_proc.c, ~/inc/def_error.h

2) log format

항목	Type	길이	예	설명
process ID	char,	15+2	pa_1101_ts ,	log write하는 process의 ID
log 시각	char,	15+1	06:00:11-041988,	HH:MM:SS-mmmmmm; mmm in microsec (1/1,000,000 sec)
유형	char,	3+1	PRO,	USR:user, SYS:system, PRO:process, SAM:file, FIF:FIFO, QUE:message queue, TCP:TCP/IP, X25:X.25, DSH:data, SHM
장애수준	char]	4+1	INFO]	INFO:normal, FATL:fatal error, DBUG:test debug, WARN:warning, EROR:non-critical error
message	char	max 5078		message 내용 중 LF (0x0a) 값은 '\$'로 치환
line feed	char	1		LF (0x0a)

3) color attribution (log_proc.c 참조)

- PZ_emergency 에 write 시 장애수준에 따라 다른 색으로 display 되도록 함
 - FATL → red (동시에 beep 경고음도 발생)
 - EROR → magenta (동시에 beep 경고음도 발생)
 - DBUG → green
 - WARN → yellow

4) log save

- 각 부문 log directory 의 process 별 log file 은 60 MB 를 넘으면 <process 명>.<hhmmss>로 save 하고 다시 logging 시작함

3.5. 주요 운영 PROCESS

1) pz_fepp_mp

- pz_procchk_mp 와 부문 daemon (pa[~pv]_daemon_mp)과 업무접속 daemon (pw_daemon_mp)을 기동하는 최 상위 daemon process 로 user 에 의해 최초 기동된 이후 항상 실행 상태인 daemon
- 부문별 daemon 기동 시각이 되면 daemon.ini 에 해당 부문의 Status 가 '1'로 되어 있는 부문의 경우 해당 부문 daemon 을 실행시킴

2) pz_compact_mp

- 매일 23:50 경 pz_procchk_mp 에 의하여 실행되어 pw ~ pz 부문의 log 와 pz 부문의 data 를 정리한 후 종료
- 각 부문 daemon 기동 시 부문 daemon 에 의하여 기동되어 해당 부문의 data 와 log 를 정리한 후 종료

3) pz_memory_mp

- pz_fepp_mp 최초 기동 시 실행되어 전체 daemon SHM (SHM_All_Daemon_Info, 즉 INFO)을 생성하고 daemon 환경 설정 file (~/cfg/daemon.ini)에 등록된 내용을 전체 daemon SHM 에 load 한 후 종료
- 각 부문 daemon 기동 시 부문 daemon 에 의하여 기동되어 해당 부문의 부문 SHM (Shm_Mem)을 생성하고 환경 설정 file (~/cfg/*.ini)에 등록된 내용을 부문 SHM 에 load 한 후 종료

4) pz_procchk_mp

- pz_fepp_mp 에 의해 최초 기동된 이후 항상 실행 상태인 daemon
- 처리 내용
 - 1) 부문 daemon 및 업무 process 의 정상 실행 여부를 check 하여 미 실행 process 가 있으면 경고 message 를 logging
 - 2) disk space (%used)를 check 하여 90%를 넘으면 경고 message 를 logging
 - 3) 각 부문 종료 시각 10 분 전에 해당 부문의 file read/write count 를 file 에 기록
(px_chkcnt_mp; fepp3/SEQ/TOTAL/totalseq.yyyymmdd)

5) pa[~pv]_daemon_mp

- 업무 부문 daemon process
- 각 부문 start time 에 기동하여 end time 에 종료 (환경 설정 file 참조)

- 최초 기동 시 `pz_memory_mp` 를 실행하여 부문 SHM 생성하고 환경을 load
- `pz_compact_mp` 를 실행하여 해당 부문의 data 및 log 를 정리
- 각 업무 process 별 start time 에 해당 process 들을 기동시키고 process 의 종료 시각이 되면 종료 처리하며 부문 end time 이 되면 자신도 종료

6) `pz_filechk_mp`

- 각 부문 daemon 기동 시 부문 daemon 에 의해 실행되어 해당 부문의 모든 data file 을 check 하여 정상이면 0, 오류 있으면 1 을 되돌려주며 종료
- 부문 daemon 은 1 을 되돌려 받았을 경우 경고 message 를 log 에 write 하고 30 초 간격으로 `pz_filechk_mp` 를 실행하여 정상일 때까지 반복함
- 운영자는 위 오류 발견 시 해당 data file 을 복구 또는 삭제하여 부문 daemon 이 정상 처리되도록 함

4. PROCESS 운영 시간

4.1. REAL SYSTEM 운영 시간

IBK Bank(PA/OMS) – 04:30 ~ 04:00				
Process ID	업무	업무그룹	업무개시/종료	Notes
pa_1201_mp	채권 응답분배	응답/체결/미체결 미니원장	07:00 ~ 19:00	당일운영
pa_1291_mp	채권 미체결 미니원장 관리		07:00 ~ 19:00	
pa_1401_mp	채권 체결관리		07:00 ~ 19:00	
pa_1491_mp	채권 체결/잔고 미니원장 관리		07:00 ~ 19:00	
pa_7102_dd	채권 시세(A3/G7/B6) 관리	시세	05:00 ~ 19:00	
pa_7103_dd	채권 M4(미사용)		05:00 ~ 19:00	
pa_7801_dd	채권 배치(RDS01) 관리		17:00 ~ 03:30	
pa_7802_dd	채권 배치(RDS02) 관리		17:00 ~ 03:30	
pa_7181_dd	채권 배치(RDS01=>A0 추출) 관리		05:00 ~ 03:30	
pa_7182_dd	채권 배치(RDS02=>A0 추출) 관리		05:00 ~ 03:30	
pa_7701_tr	채권시세수신(From.시세 FEP)		06:00 ~ 23:00	
pa_7702_tr	채권기타시세수신(From.시세 FEP)		06:00 ~ 23:00	
pa_7622_tr	파생시세수신(유진)		07:30 ~ 23:00	
pa_7623_tr	파생기타시세수신(유진)		07:30 ~ 23:00	
pa_8101_ts	내부 송신_전략기동/종료등 결과	접속서버 수신	06:00 ~ 23:00	
pa_8105_ts	내부 송신_채권장운영_업무		06:00 ~ 23:00	
pa_8111_ts	내부 송신_채권체결 DP 송신		06:00 ~ 23:00	
pa_8116_ts	내부 송신_일괄수신 응답전문		07:00 ~ 02:00	
pa_8117_ts	내부 송신_RDS01_KRX 데이터		06:00 ~ 23:00	
pa_8118_ts	내부 송신_RDS02_KRX 데이터		06:00 ~ 23:00	
pa_8201_tr	내부 수신_전략기동/종료등 요청		06:00 ~ 23:00	
pa_8211_tr	내부 수신_일괄수신 KRX 전송전문		06:00 ~ 23:00	
pa_9001_mp	관리용 메모리 초기화	관리용	04:00 ~ 23:00	
pa_50101mp	자동전략	자동전략	08:00 ~ 19:00	
pa_50102mp	자동전략		08:00 ~ 19:00	
pa_50103mp	자동전략		08:00 ~ 19:00	
pa_50104mp	자동전략		08:00 ~ 19:00	
pa_50105mp	자동전략		08:00 ~ 19:00	

pa_50106mp	자동전략		08:00 ~ 19:00	
pa_50107mp	자동전략		08:00 ~ 19:00	
pa_50108mp	자동전략		08:00 ~ 19:00	
pa_50109mp	자동전략		08:00 ~ 19:00	
pa_50110mp	자동전략		08:00 ~ 19:00	
pa_50201mp	자동전략		08:00 ~ 19:00	
pa_50202mp	자동전략		08:00 ~ 19:00	
pa_50203mp	자동전략		08:00 ~ 19:00	
pa_50204mp	자동전략		08:00 ~ 19:00	
pa_50205mp	자동전략		08:00 ~ 19:00	
pa_50206mp	자동전략		08:00 ~ 19:00	
pa_50207mp	자동전략		08:00 ~ 19:00	
pa_50208mp	자동전략		08:00 ~ 19:00	
pa_50209mp	자동전략		08:00 ~ 19:00	
pa_50210mp	자동전략		08:00 ~ 19:00	
~ pa_50410mp	자동전략 (총 40 개의 자동전략이 있음)		08:00 ~ 19:00	
px_showsize _mp	메모리 조회	메모리 조회	상시	

IBK Bank(PB/시세) – 04:30 ~ 04:00				
Process ID	업무	업무그룹	업무개시/종료	Notes
pb_7102_ur	채권 시세(A3/G7/B6) 수신	시세 FEP	07:00 ~ 19:00	당일운영
pb_7102_ur	채권 기타(M4) 수신 (미사용)		07:00 ~ 19:00	
pb_7102_dd	채권 시세 메모리처리		07:00 ~ 19:00	
pb_7102_dd	채권 기타(M4) 메모리처리 (미사용)		07:00 ~ 19:00	
pb_7612_ts	채권 OMS 시세전송		07:00 ~ 19:00	
pb_7613_ts	채권 OMS 기타전송 (미사용)		07:00 ~ 19:00	

IBK Bank(PB/주문) – 04:30 ~ 04:00				
Process ID	업무	업무그룹	업무개시/종료	Notes
pb_1101_ts	채권 주문송신	주문 FEP	07:00 ~ 19:00	당일운영
pb_2101_ts	파생 주문송신		07:00 ~ 19:00	
pb_1201_tr	채권 응답/채결수신		07:00 ~ 19:00	
pb_2201_tr	파생 응답/채결수신		07:00 ~ 19:00	
pb_1211_tr	채권 DropCopy 수신		07:00 ~ 19:00	
pb_1601_tr	채권 전체장운영 수신		07:00 ~ 19:00	
pb_7801_tr	일괄수신 RDS 수신		07:00 ~ 19:00	
pb_1801_ts	일괄수신(지점정보)		07:00 ~ 19:00	

5. 운영 UTILITY SHELL 사용법

5.1. 조회 관련 SHELL 사용법

Name	내용	사용법
chkcnt.sh	process 일련번호 및 file read/write 건수 조회	chkcnt.sh all; chkcnt.sh PA; chkcnt.sh PA PA_1101_ts; chkcnt.sh PA PA; chkcnt.sh PA batch; chkcnt.sh PA proc; chkcnt.sh PA file
chkproc.sh	process 정보 (PROC) 조회	chkproc.sh PA; chkproc.sh PA tr
chkshm.sh	shared memory 상세 조회 (file 생성)	chkshm.sh
chktcp1.sh	TCP/IP (port daemon 방식) 프로세스 line 상태 조회	chktcp1.sh PA; chktcp1.sh all
chktcp2.sh	TCP/IP 직접 접속 프로세스 line 상태 조회	chktcp2.sh PA; chktcp2.sh all

5.2. 개발 관련 SHELL 사용법

Name	내용	사용법
cfgback.sh	환경설정 file 수정용 file 생성 (.ini → .ini.tmp)	cfgback.sh tcp1; cfgback.sh all
cfgload.sh	환경설정 file 수정 후 반영 (.ini.tmp → .ini)	cfgload.sh tcp1; cfgload.sh all
mk.sh	source program 의 compile	mk.sh all; mk.sh sub; mk.sh PA; mk.sh PA ts; mk.sh src
mkall.sh	source program 의 compile (오류 display)	mkall.sh all; mkall.sh sub; mkall.sh PA

5.3. 변경 관련 SHELL 사용법

Name	내용	사용법
jstop.sh	process 종료	jstop.sh PA_1101_ts
runstop.sh	부문 daemon 기동 및 강제 종료 처리	runstop.sh
setcseq.sh	process 의 counter sequence 변경	setcseq.sh PA_1201_ts 10
setdate.sh	업무 process 들의 운영일자 변경	setdate.sh PA 20071029
setdname.sh	process 의 in/out data SHM 또는 fifo name 변경	setdname.sh PA_1201_dd I1 PA_1201_dd PA_1201_dd1; setdname.sh PA_1201_dd O1 PA_1201_ts
setdsize.sh	data SHM 의 data size 변경	setdsize.sh PA_1101_ts 198
setdtime.sh	부문 start/end time, date flag 변경	setdtime.sh PA 0530 0300 2
setfcnt.sh	file 의 read/write 건수 변경	setfcnt.sh PA_1101_ts R1 0
setfname.sh	process 의 input file 명/FIFO 명, output file 명 변경	setfname.sh PA_1401_ts I1 PA_1401_ts PA_1401_ts1 setfname.sh PA_1401_tr O1 PA_1401_dd
setfsize.sh	file 의 data size 변경	setfsize.sh PA_1401_ts 210
setinfostat.sh	부문 daemon 의 process status 변경	setinfostat.sh PA 1
setlstat.sh	X.25, TCP2 process 의 line status 변경	setlstat.sh PA_1101_ts P 1
setnstat.sh	X.25, TCP process 의 network status 변경	setnstat.sh PA_1101_ts P 1
setpseq.sh	process 의 interface sequence 변경	setpseq.sh PA_1101_ts 10
setpstat.sh	process 의 process status 변경	setpstat.sh PA_1101_ts 1; setpstat.sh PAts 1
setptime.sh	process 의 start/end time 변경	setptime.sh PA_1101_tr s 0630
setsstat.sh	process 의 start status (업무상태) 변경	setsstat.sh PA_1101_tr 3; setsstat.sh PAtr 1
sett2ip.sh	TCP2 client process 의 server IP address 변경	sett2ip.sh PA_1101_ts P 200.100.3.191

sett2seq.sh	TCP2 client process 의 일련번호를 server 측에 맞추어 변경	sett2seq.sh PA; sett2seq.sh PA PA_1101_ts
settout.sh	process 의 timeout 값 변경	settout.sh PA_1201_ts 10

Name	내용 및 사용법
psk.sh	process kill 처리 [용법] psk.sh <process name> <option> - process name: 해당 string 을 포함하는 process 모두 찾을 - option: USR1, USR2, 9, 없음 1. “USR1” → process 기동 1) 부문 daemon (pa[~pw]_daemon_mp)에만 사용 2) signal 을 받은 부문 daemon 의 하위 업무 process 기동 2. “USR2” → process 종료 1) 부문 daemon (pa [~pw]_daemon_mp)에만 사용 (이 경우는 “USR1”으로 업무 process 기동 가능) 2) FEP 전 process 를 종료시킬 경우 최상위 daemon (pz_fepp_mp)을 “USR2”로 kill 하면 됨 3. “9” → 강제 종료 1) 해당 process 자신만 종료 (가급적이면 jstop.sh 을 이용하여 정상적으로 종료시킬 것) 2) 상위 daemon 에 의하여 자동 기동되지 않음 3) “USR1”으로 기동할 수 없음 4. option 없음 (SIGTERM (15)) 1) process.ini 에 해당 process 의 Status 가 ‘R’로 된 경우 process 종료 직후 재기동됨 2) process.ini 에 Status 가 ‘S’로 되어 있는 경우 해당 process 종료 후 재기동되지 않음 3) 최상위 daemon (pz_fepp_mp)의 경우는 pz_fepp_mp 만 재기동됨